

The Data Science Interview Handbook — Volume 2

50 Interview Questions Decoded: Fundamentals to Advanced ML

A premium interview preparation guide covering Data Science foundations, statistics, machine learning theory, and deep learning — answered simply, with real examples, and structured for quick revision.

 **How to use this handbook:** Every question follows the same structure. When you read an answer, don't memorize it — understand *why* each step matters and practice explaining it in your own words. Use the **Quick Revision** box before interviews.

Table of Contents

Part I — Data Science Fundamentals

#	Topic
1	Types of Data Science Problems
2	Common Issues in Raw Datasets
3	Learning Mechanisms in Data Science
4	Standard Deviation vs. Variance
5	Overfitting and Underfitting
6	Regularization in Machine Learning
7	Testing Model Generalization
8	Role of Activation Functions
9	Confusion Matrix + FP vs. FN
10	Decision Tree vs. Random Forest

Part II — Statistics & Probability

#	Topic
11	Naive in Naive Bayes
12	Imbalanced Data & SMOTE
13	Statistical Tests & p-Value
14	Eigenvalues and Eigenvectors
15	Sampling & Representativeness
16	Gradient and Gradient Descent
17	Confounding Variables
18	Bias-Variance Trade-off
19	Central Limit Theorem
20	Convex vs. Non-Convex Functions

Part III — Advanced Data Science

#	Topic
21	Collaborative Filtering
22	Lazy Learning Algorithms
23	Bagging vs. Boosting
24	When to Use Deep Learning
25	OOB Error in Random Forest
26	How to Choose a Learning Algorithm
27	Feature Scaling
28	Handling Missing Values
29	A/B Testing
30	Feature Selection

Part IV — Machine Learning Core

#	Topic
31	What is Machine Learning
32	What is a Model in ML
33	Cross-Validation
34	Precision and Recall
35	F1 Score
36	Feature Engineering
37	PCA — Principal Component Analysis
38	Support Vector Machines (SVM)
39	Dropout in Neural Networks
40	Convolutional Neural Networks (CNN)

Part V — Advanced ML & Bonus Questions

#	Topic
41	Recurrent Neural Networks for Time Series
42	Hyperparameter Tuning
43	Reinforcement Learning
44	Type I vs. Type II Errors
45	ROC Curve and AUC
46	Transfer Learning
47	Vanishing Gradient Problem
48	Correlation vs. Causation
49	K-Means Clustering
50	XGBoost Explained

PART I — DATA SCIENCE FUNDAMENTALS

Question 1: Types of Data Science Problems

The Interview Question

"Explain the types of Data Science problems along with datasets typically used for each problem type."

Why Interviewers Ask This

- To check whether you understand the landscape of ML — not just one type
 - To see if you can map a business problem to the correct problem type
 - To evaluate your ability to match datasets to techniques
-

Complete Interview Answer

Every real-world data science problem falls into one of several fundamental categories. Identifying the correct type is the *first* step before choosing any model.

The 5 Core Problem Types

Problem Type	What It Predicts	Example Dataset	Example Algorithm
Classification	A category / class label	Fraud vs. Not Fraud transactions	Logistic Regression, Random Forest
Regression	A continuous numeric value	House prices	Linear Regression, XGBoost
Time Series Forecasting	Future values from historical sequences	Stock prices, monthly sales	ARIMA, LSTM
Recommendation System	What a user would prefer	Movie ratings, purchase history	Collaborative Filtering, Matrix Factorization
Clustering	Natural groupings with no labels	Customer demographics (age, income, habits)	K-Means, DBSCAN

Step 1 — Classification

- The goal is to predict **which category** an input belongs to
- Output is a **discrete label**, not a number
- Example: Given transaction details → predict *Fraud* or *Not Fraud*

Step 2 — Regression

- The goal is to predict a **continuous numeric outcome**
- Example: Given house size, location, and age → predict *price in dollars*

Step 3 — Time Series Forecasting

- Data is **ordered by time** and the model learns temporal patterns
- Example: Given 5 years of daily stock prices → predict tomorrow's price

Step 4 — Recommendation Systems

- The goal is to predict what a user will **like or purchase next**
- Example: Netflix suggests movies based on your watch history

Step 5 — Clustering

- There are **no labels** — the model finds natural groupings in data
- Example: Group customers by age, income, and spending habits → discover distinct market segments



Python Example

```
python
```

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.linear_model import LinearRegression
from sklearn.cluster import KMeans

# --- Classification example: Fraud Detection ---
# Features: transaction_amount, time_of_day, location
# Label: is_fraud (0 or 1)
clf = LogisticRegression()
# clf.fit(X_train, y_train)

# --- Regression example: House Pricing ---
# Features: size, bedrooms, age
# Label: price (a number)
reg = LinearRegression()
# reg.fit(X_train, y_train)

# --- Clustering example: Customer Segmentation ---
# Features: age, income, purchase_frequency
# No labels needed!
km = KMeans(n_clusters=4, random_state=42)
# km.fit(X)
```

✅ Important Concepts

- ✅ Classification = discrete output (labels)
- ✅ Regression = continuous output (numbers)
- ✅ Time Series = sequential, time-ordered data
- ✅ Recommendation = user-item interaction data
- ✅ Clustering = unsupervised (no labels)

🧰 Python Libraries

Library	Purpose
scikit-learn	Classification, Regression, Clustering
statsmodels	Time Series (ARIMA)
pandas	Dataset handling
surprise	Recommendation systems

💡 Interview Tip

💡 **Interview Tip:** Always start your answer by identifying the problem type before naming any algorithm. Saying "*I would first determine whether this is a classification or regression problem*" immediately signals senior-level thinking.

❌ Common Mistakes

- ❌ Jumping straight to an algorithm without identifying the problem type
 - ❌ Confusing regression with classification (both predict — but very different outputs)
 - ❌ Forgetting that clustering has **no target variable**
-

💬 Follow-up Questions

- How would you handle a problem that is both classification and regression?
 - What if you have no labeled data — what problem type does that fall into?
 - How would you frame "predict customer lifetime value" as a problem type?
-

🕒 30-Second Interview Answer

"Data science problems fall into five main types: classification predicts a category like fraud or not-fraud; regression predicts a number like house price; time series forecasting predicts future values from historical sequences; recommendation systems predict what a user wants next; and clustering groups data without labels. The problem type determines the algorithm choice."

🧑 2-Minute Interview Answer

"The first thing I do when approaching any data science project is identify the problem type, because that determines every subsequent decision — data collection, model selection, and evaluation metrics. The five core types are classification, regression, time series forecasting, recommendation systems, and clustering. Classification predicts a discrete label — fraud detection is a classic example where each transaction is labeled as fraud or not fraud. Regression predicts a continuous number — house price prediction uses features like size and location to output a price in

dollars. Time series forecasting handles sequential, time-ordered data, like predicting stock prices or monthly sales using historical patterns. Recommendation systems predict what a user would prefer next based on past behavior, like Netflix or Amazon product suggestions. Clustering is the only unsupervised problem type — there are no labels; instead, the model discovers natural groupings, like segmenting customers by age, income, and purchasing habits.

Identifying the problem type correctly upfront prevents a lot of mistakes downstream — if you model a regression problem as classification, you lose granular information in the predictions."

Quick Revision

- Classification = labels → Logistic Regression, Random Forest
 - Regression = numbers → Linear Regression, XGBoost
 - Time Series = time-ordered → ARIMA, LSTM
 - Recommendation = user preferences → Collaborative Filtering
 - Clustering = no labels → K-Means, DBSCAN
-
-

Question 2: Common Issues in Raw Datasets That Require Cleaning

The Interview Question

"What are the common issues in raw datasets that require data cleaning? How would you handle each one?"

Why Interviewers Ask This

- To test your practical data-wrangling experience
 - To see if you know *which* technique to use for *which* problem
 - To evaluate whether you handle cleaning systematically vs. randomly
-

Complete Interview Answer

Raw real-world data is almost never ready for modeling. The most common issues — and their treatments — are:

Issue 1 — Missing Values

The most common problem. Treatment depends on data type and distribution:

Column Type	Condition	Treatment
Numerical	No significant outliers	Fill with mean
Numerical	Outliers present	Fill with median (outlier-resistant)
Categorical	Any	Fill with mode (most frequent value)
Any	> 50-70% missing	Consider dropping the column

Issue 2 — Duplicate Rows

- Duplicates inflate counts and bias models
- Detection: `df.duplicated().sum()`
- Fix: `df.drop_duplicates(inplace=True)`

Issue 3 — Outliers

- Outliers can pull the mean, distort regression lines, and confuse clustering
- Detection: box plots, Z-score, IQR method
- Treatment:
 - **Standardization** (Z-score): $(x - \text{mean}) / \text{std}$ → mean 0, std 1
 - **Normalization** (Min-Max): $(x - \text{min}) / (\text{max} - \text{min})$ → range 0 to 1
 - Cap at a threshold (Winsorization)
 - Remove if confirmed as data error

Issue 4 — Inconsistent Formatting

- Mixed date formats: `"01/01/2024"` vs. `"2024-01-01"`
- Mixed case: `"London"` vs. `"london"` vs. `"LONDON"`
- Fix with `pd.to_datetime()`, `.str.lower()`, `.str.strip()`

Issue 5 — Incorrect Data (Domain Violations)

- A negative age, a temperature of 500°C, a salary of -£5,000
 - These are not outliers — they are **impossible values**
 - Fix: set a domain-valid range and either correct or remove the offending rows
-

Python Example

python

```
import pandas as pd
import numpy as np

df = pd.read_csv('raw_data.csv')

# Issue 1: Missing values
print(df.isnull().sum()) # count missing per column
df['salary'].fillna(df['salary'].median(), inplace=True) # numerical → median
df['city'].fillna(df['city'].mode()[0], inplace=True) # categorical → mode

# Issue 2: Duplicates
print(f"Duplicates: {df.duplicated().sum()}")
df.drop_duplicates(inplace=True)

# Issue 3: Outliers – IQR method
Q1, Q3 = df['age'].quantile(0.25), df['age'].quantile(0.75)
IQR = Q3 - Q1
df = df[(df['age'] >= Q1 - 1.5 * IQR) & (df['age'] <= Q3 + 1.5 * IQR)]

# Issue 4: Inconsistent formatting
df['date'] = pd.to_datetime(df['date'], errors='coerce')
df['city'] = df['city'].str.strip().str.title()

# Issue 5: Domain violations – negative age is impossible
df = df[df['age'] > 0]
```

✓ Important Concepts

- ✓ Mean imputation (no outliers) vs. median imputation (outliers present)
 - ✓ Mode for categorical columns
 - ✓ IQR method for outlier detection
 - ✓ `drop_duplicates()` for duplicate rows
 - ✓ Domain validation for impossible values
-

Python Libraries

Library	Purpose
pandas	Cleaning, imputation, deduplication
numpy	Statistical calculations
missingno	Visualizing missing patterns
seaborn / matplotlib	Box plots, distribution checks

Interview Tip

 **Interview Tip:** Frame your answer as a checklist — interviewers love structured thinking: *"I check for five things in order: missing values, duplicates, outliers, inconsistent formatting, and domain violations. Each has a different fix depending on context."*

Common Mistakes

-  Always using mean imputation — skewed data needs median
-  Dropping outliers without investigating if they're real or errors
-  Forgetting to check formatting issues in date or string columns
-  Treating domain violations as outliers instead of data errors

Follow-up Questions

- When should you drop a column vs. impute it?
- How do you handle missing values in a test dataset (no target labels)?
- What's the risk of removing outliers from a fraud dataset?

30-Second Interview Answer

"The five most common raw data issues are missing values, duplicates, outliers, inconsistent formatting, and impossible values. For missing numerics I use mean if data is symmetric, median if outliers exist, and mode for categoricals. Duplicates get

removed. Outliers get flagged with IQR or Z-score and treated based on context.
Formatting and domain violations are fixed with pandas string and datetime methods."

2-Minute Interview Answer

"Raw data almost always has five categories of issues that need systematic cleaning before any modeling. First, missing values — my rule of thumb is to use the mean for normally distributed numeric columns, the median when outliers are present because mean is sensitive to them, and the mode for categorical columns. If a column has more than 70% missing, I consider dropping it entirely. Second, duplicates — straightforward to detect and remove, but often overlooked, and they can artificially inflate model performance metrics. Third, outliers — I use the IQR method or Z-scores to detect them, and then I decide: are these real extreme values, or data entry errors? Fraud data might have legitimate extreme values that shouldn't be removed. Fourth, inconsistent formatting — dates in mixed formats, strings in mixed case — these cause silent bugs and wrong group counts, so I standardize with pandas methods. Fifth, domain violations like negative ages or temperatures above physical limits — these aren't outliers, they're impossible values and should be removed or corrected using domain knowledge."

Quick Revision

- Missing numerical: mean (normal) or median (skewed/outliers)
 - Missing categorical: mode
 - Duplicates: `drop_duplicates()`
 - Outliers: IQR method, Z-score → investigate before removing
 - Impossible values: remove using domain constraints
-
-

Question 3: Different Learning Mechanisms in Data Science

The Interview Question

"What are the different learning mechanisms (paradigms) in data science? Explain each one with a real-world example."

Why Interviewers Ask This

- To check foundational ML knowledge

- To see if you understand the *difference in how* each paradigm learns, not just names
 - To evaluate whether you can connect each type to a practical scenario
-

🌟 Complete Interview Answer

Overview Table

Learning Paradigm	Has Labels?	How It Learns	Real-World Example
Supervised	✅ Yes	Learns from labeled input-output pairs	Email spam classifier
Unsupervised	❌ No	Finds hidden structure in unlabeled data	Customer segmentation
Semi-Supervised	⚡ Some	Uses a small labeled set + large unlabeled set	Image tagging with limited labels
Reinforcement	🏆 Reward signal	Learns by trial and error in an environment	Game-playing AI (AlphaGo), robot navigation

Supervised Learning

The model is trained on **labeled data** — each input has a known correct output. The model learns to predict labels for new, unseen inputs.

- Example: Training on thousands of emails labeled *spam* or *not spam*, then classifying new emails
- Algorithms: Logistic Regression, Random Forest, Neural Networks

Unsupervised Learning

There are **no labels**. The model discovers patterns, groupings, or structure in the data on its own.

- Example: Grouping customers into segments based on age, income, and purchase frequency — no one tells the model how many groups or what they mean
- Algorithms: K-Means, DBSCAN, PCA

Semi-Supervised Learning

A hybrid approach — a **small labeled dataset** combined with a **large unlabeled dataset**. Labeling is expensive, so this approach saves cost while leveraging more data.

- Example: Training a medical image classifier with 500 labeled X-rays and 10,000 unlabeled ones
- The model uses the unlabeled data to learn better representations

Reinforcement Learning

The model (called an **agent**) learns by **interacting with an environment**, receiving rewards for correct actions and penalties for wrong ones — no labeled dataset, just feedback over time.

- Example: A robot learning to walk — every time it doesn't fall, it gets a reward; every time it falls, it gets a penalty
- Example: Google's AlphaGo mastered the board game Go through millions of self-played games

Python Example

```
python

from sklearn.linear_model import LogisticRegression      # Supervised
from sklearn.cluster import KMeans                    # Unsupervised

# SUPERVISED: Spam detection
# X = email features, y = 0 (not spam) or 1 (spam)
supervised_model = LogisticRegression()
# supervised_model.fit(X_train, y_train)

# UNSUPERVISED: Customer segmentation
# X = customer features, no labels needed
unsupervised_model = KMeans(n_clusters=4, random_state=42)
# unsupervised_model.fit(X)

# REINFORCEMENT: Use OpenAI Gym for RL environments
import gym # provides simulated environments for RL agents
# env = gym.make('CartPole-v1')
```

✅ Important Concepts

- ✅ Supervised = labeled data, predict output
- ✅ Unsupervised = no labels, discover structure
- ✅ Semi-supervised = mostly unlabeled, few labels
- ✅ Reinforcement = reward/penalty from environment

💡 Interview Tip

💡 **Interview Tip:** Connect each type to *why* it exists: "Supervised learning is most common when labels exist and are affordable. Semi-supervised exists because labeling is expensive. Unsupervised is used when labeling is impossible. Reinforcement exists for sequential decision-making problems where there is no dataset — only experience."

❌ Common Mistakes

- ❌ Forgetting semi-supervised as a distinct category
 - ❌ Describing reinforcement learning as simply "unsupervised" — they are fundamentally different
 - ❌ Confusing the reward signal in RL with supervised labels
-

🔄 Follow-up Questions

- What happens when you have 99% unlabeled and 1% labeled data?
 - Give an example where supervised learning would fail and unsupervised would be needed
 - Is self-supervised learning different from semi-supervised?
-

🕒 30-Second Interview Answer

"There are four learning paradigms. Supervised learning uses labeled data to predict outcomes, like spam detection. Unsupervised learning finds hidden patterns in unlabeled data, like customer segmentation. Semi-supervised uses a mix of labeled and unlabeled data. Reinforcement learning trains an agent through rewards and penalties in an environment, like game-playing AI."

📌 Quick Revision

- Supervised → labeled → predict → spam filter, house prices
 - Unsupervised → no labels → discover groups → clustering
 - Semi-supervised → few labels + many unlabeled → cost-efficient
 - Reinforcement → reward/penalty → agent learns → AlphaGo, robotics
-

Question 4: Why is Standard Deviation Preferred Over Variance?

The Interview Question

"Why is standard deviation often preferred over variance when analyzing and communicating data?"

Why Interviewers Ask This

- Tests basic but important statistical intuition
 - Checks whether you understand *practical usability*, not just formulas
 - Often asked as a warm-up to deeper statistics questions
-

Complete Interview Answer

Both **variance** and **standard deviation** measure how spread out data points are from the mean. But they differ in one crucial way.

The Key Difference

Metric	Formula	Unit	Interpretation
Variance	Mean of squared deviations from mean	Squared units (e.g., cm ²)	Hard to relate to original data
Standard Deviation	Square root of variance	Same unit as data (e.g., cm)	Directly comparable to data values

Simple Example

Imagine student exam scores: [70, 75, 80, 85, 90]

- Mean = 80
- Deviations from mean: [-10, -5, 0, 5, 10]
- Squared deviations: [100, 25, 0, 25, 100]
- **Variance** = (100+25+0+25+100) / 5 = **50 score²** ← meaningless unit
- **Standard Deviation** = $\sqrt{50} \approx 7.07$ **scores** ← meaningful: average exam scores are ~7 points from the mean

Why Standard Deviation Wins in Practice

1. **Same unit as the data** — easy to interpret. If height is measured in cm, SD is also in cm
2. **Human-friendly** — telling a stakeholder "sales vary by $\pm \$500$ on average" is far more intuitive than "variance is 250,000 dollars squared"
3. **Used in most formulas** — Z-scores, confidence intervals, and normal distributions all use SD
4. **Works with the 68-95-99.7 rule** — in a normal distribution, ~68% of data falls within 1 SD, ~95% within 2 SDs

✦ **Rule of Thumb:** Use standard deviation for communication and real-world interpretation. Variance is used internally in mathematical derivations (e.g., ANOVA uses variance because it's additive across independent sources).

Python Example

```
python

import numpy as np

scores = [70, 75, 80, 85, 90]

mean = np.mean(scores)
variance = np.var(scores)
std_dev = np.std(scores)

print(f"Mean: {mean}")
print(f"Variance: {variance:.2f} (units: score²)")
print(f"Standard Deviation: {std_dev:.2f} (units: score)")

# 68-95-99.7 Rule check
print(f"Most students scored between {mean - std_dev:.1f} and {mean + std_dev:.1f}")
```

Output:

```
Mean: 80.0
Variance: 50.00 (units: score²)
Standard Deviation: 7.07 (units: score)
Most students scored between 72.9 and 87.1
```

✓ Important Concepts

- ✓ Both measure spread — but in different units
 - ✓ Variance = SD^2 (squared units → harder to interpret)
 - ✓ Standard deviation is in the same unit as the original data
 - ✓ 68-95-99.7 rule uses standard deviation
-

💡 Interview Tip

💡 **Interview Tip:** Always give a concrete example when explaining statistics. Saying "*if we're measuring weights in kg, variance is kg^2 which is meaningless — standard deviation keeps us in kg*" is much more memorable than a formula.

✗ Common Mistakes

- ✗ Saying variance is "wrong" — it's mathematically correct but less interpretable
 - ✗ Forgetting that variance is used in ANOVA and mathematically convenient contexts
-

🔗 Follow-up Questions

- When would you deliberately use variance instead of standard deviation?
 - What does a high vs. low standard deviation tell you about a dataset?
 - How does standard deviation relate to the normal distribution?
-

🕒 30-Second Interview Answer

"Both measure spread, but variance squares the deviations, so its unit is also squared — if you're measuring height in cm, variance is in cm^2 , which is hard to interpret. Standard deviation takes the square root, bringing the unit back to cm, making it directly comparable to the original data. That's why we prefer it for communication and most real-world analysis."

📌 Quick Revision

- Both = measure of spread
- Variance = squared units (hard to interpret)

- Standard Deviation = same unit as data (interpretable)
 - SD used in Z-scores, confidence intervals, normal distribution
-
-

Question 5: Overfitting and Underfitting — Detection and Treatment

The Interview Question

"What is overfitting and underfitting? How can you detect if your model is suffering from either, and what steps would you take to fix each?"

Why Interviewers Ask This

- One of the most frequently asked ML theory questions
 - Tests understanding of the bias-variance trade-off
 - Evaluates practical debugging skills
-

Complete Interview Answer

The Core Concept

Condition	What It Means	Analogy
Underfitting	Model is too simple — can't even learn the training data	A student who memorized nothing before an exam
Good Fit	Model generalizes well to new data	A student who understood the concepts
Overfitting	Model memorizes training data — fails on new data	A student who memorized answers but can't handle new questions

How to Detect Overfitting

- **Training accuracy is very high, but test/validation accuracy is significantly lower**
- Example: Training accuracy = 98%, Test accuracy = 72% → the gap is the overfitting signal

Training Error: ▼▼▼ (very low)
Validation Error: ▲▲▲ (much higher)

How to Detect Underfitting

- **Both training and test accuracy are low**
- Example: Training accuracy = 55%, Test accuracy = 53% → both poor

Training Error: ▲▲▲ (high)
Validation Error: ▲▲▲ (also high)

Fixing Overfitting

1. **Reduce the number of features** — remove irrelevant inputs
2. **Apply regularization** — L1 (Lasso) or L2 (Ridge) penalizes complexity
3. **Use cross-validation** — k-fold CV gives a fairer picture of generalization
4. **Get more training data** — more data makes it harder to memorize noise
5. **Use dropout** — for neural networks, randomly deactivate neurons during training

Fixing Underfitting

1. **Add more features** — give the model more signal to learn from
2. **Increase model complexity** — switch from linear regression to decision trees, or add layers to a neural network
3. **Reduce regularization** — overly strong regularization can cause underfitting
4. **Train for longer** — especially relevant for neural networks

Python Example

```
python
```

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import Ridge
from sklearn.tree import DecisionTreeClassifier
import numpy as np

# Detecting: compare training vs. test performance
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

model = DecisionTreeClassifier(max_depth=None) # unlimited depth → risks overf
model.fit(X_train, y_train)

train_acc = model.score(X_train, y_train)
test_acc = model.score(X_test, y_test)

print(f"Training accuracy: {train_acc:.3f}")
print(f"Test accuracy: {test_acc:.3f}")
print(f"Gap (overfitting?): {train_acc - test_acc:.3f}") # large gap = overfi

# Fixing overfitting: use cross-validation + regularized model
cv_scores = cross_val_score(Ridge(alpha=1.0), X, y, cv=5)
print(f"CV mean: {np.mean(cv_scores):.3f} ± {np.std(cv_scores):.3f}")
```

✓ Important Concepts

- ✓ Overfitting: high training accuracy, low test accuracy
- ✓ Underfitting: low accuracy on both sets
- ✓ Fix overfitting: regularization, fewer features, more data, cross-validation
- ✓ Fix underfitting: more features, higher complexity, less regularization

💡 Interview Tip

💡 **Interview Tip:** Always tie detection to *numbers*: "I'd compare training accuracy against validation accuracy — a gap greater than 10–15% is usually a red flag for overfitting."

✗ Common Mistakes

- ✗ Saying "overfitting means low accuracy" — it means *inconsistent* accuracy (high train, low test)

- ❌ Fixing overfitting by adding more features (that makes it worse)
 - ❌ Not using cross-validation, relying on a single train/test split
-

Follow-up Questions

- What does a learning curve tell you about overfitting vs. underfitting?
 - Can a model simultaneously overfit one subset of features and underfit another?
 - What is the "sweet spot" in the bias-variance trade-off?
-

30-Second Interview Answer

"Overfitting is when a model performs well on training data but poorly on unseen data — it has memorized the training set rather than learning general patterns. I detect it by comparing training vs. test accuracy: a large gap signals overfitting. I fix it with regularization, cross-validation, or reducing features. Underfitting is when performance is poor on both sets — the model is too simple. I fix that by adding features or increasing model complexity."

Quick Revision

- Overfit: train high, test low → regularize, reduce features, get more data
- Underfit: train low, test low → more features, more complex model
- Detection tool: training vs. validation accuracy gap + learning curves

MDEOF echo "Q1-5 created"